E-Ticaret Satış Analizi: PostgreSQL Primary–Replica, Python ve Grafana

Ubuntu / VirtualBox üzerinde sıfırdan uygulama rehberi — 30 Ekim 2026

1. Hedef ve mimari

Bu proje, PostgreSQL üzerinde gerçek streaming replication kullanır. Python orders tablosunu Replica'dan okur, günlük satış istatistiklerini hesaplar ve Primary'ye yazar. Grafana yalnızca Replica'ya bağlanır.

- Ubuntu host: Primary 127.0.0.1:5432, Replica 127.0.0.1:5433, Grafana localhost:3000.
- Docker içi: Primary postgres-primary:5432, Replica postgres-replica:5432.
- Replikasyon: Primary → WAL → Replica. Replica read-only çalışır.
- Bu iki PostgreSQL container'ı aynı VM üzerinde CPU/RAM/disk kaynaklarını paylaşır. Port ayırmak tek başına performans kazancı değildir; farklı instance'lar sorgu yükünü mantıksal olarak ayırır.
- Asenkron replikasyon kısa gecikmelere yol açabilir. Bu eğitim projesi üretim ortamı güvenliği veya otomatik failover sağlamaz.

2. Eski projeyi güvenle durdur

Eski verileri korumak için eski klasörü ve volume'ları silme. Eski container çalışıyorsa önce yedek al:

```
mkdir -p ~/ecommerce-backups

sudo docker exec ecommerce-postgres pg_dump -U ecommerce_user -d ecommerce_db -Fc > ~/ecommerce-backups/old_
ecommerce.dump

ls -lh ~/ecommerce-backups/old_ecommerce.dump
```

Eski proje klasöründen kapat:

```
cd ~/ecommerce-analytics

sudo docker compose down
```

docker compose down -v, docker volume prune ve docker system prune --volumes komutlarını kullanma. Yeni proje ayrı klasör, Compose proje adı ve volume'larla çalışacak. Önceki projeyi daha sonra kaldırabilirsin.

3. Ön koşullar ve port kontrolü

```
docker --version

docker compose version

sudo systemctl is-active docker

sudo ss -lntp | grep -E ':(5432|5433|3000)\b' || true

sudo docker ps -a
```

Docker servisi active değilse sudo systemctl start docker dene. Portlar doluysa hangi programın kullandığını araştır; rastgele servis silme. VirtualBox VM'de en az yaklaşık 4 GB RAM ve yeterli boş disk alanı önerilir.

4. Yeni klasörleri oluştur

```
mkdir -p ~/ecommerce-replica/{primary,replica,python,grafana/provisioning/datasources}

cd ~/ecommerce-replica
```

Oluşturacağımız dosyalar:

```
ecommerce-replica/

├── compose.yaml

├── primary/
```

```
│   ├── init.sql
│   └── setup-replication.sh
├── replica/
│   ├── Dockerfile
│   └── replica-entrypoint.sh
├── python/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app.py
└── grafana/
    └── provisioning/datasources/postgres.yaml
```

Dosyaları nano DOSYA_ADI ile oluştur; kaydetmek için Ctrl+O, Enter, Ctrl+X.

5. Primary: başlangıç tabloları — primary/init.sql

```
CREATE TABLE IF NOT EXISTS orders (
    order_id SERIAL PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(12,2) NOT NULL CHECK (unit_price >= 0),
    order_date DATE NOT NULL
);

CREATE TABLE IF NOT EXISTS daily_sales_summary (
    sale_date DATE PRIMARY KEY,
    total_revenue NUMERIC(14,2) NOT NULL,
    total_quantity INTEGER NOT NULL,
    order_count INTEGER NOT NULL,
    average_order_value NUMERIC(14,2) NOT NULL
);

INSERT INTO orders (product_name, quantity, unit_price, order_date) VALUES
('Laptop',2,25000,'2026-10-20'),
('Mouse',5,500,'2026-10-20'),
('Klavye',3,1500,'2026-10-21'),
('Monitor',2,8000,'2026-10-21'),
('Kulaklik',4,1200,'2026-10-22'),
('Laptop',1,25000,'2026-10-22'),
('Mouse',8,500,'2026-10-23'),
('Monitor',1,8000,'2026-10-23'),
('Klavye',6,1500,'2026-10-24'),
('Kulaklik',3,1200,'2026-10-24');
```

Bu SQL yalnızca boş Primary veri volume'unun ilk başlangıcında otomatik çalışır. Aynı SQL'i tekrar çalıştırırsan siparişler tekrar eklenir.

6. Primary: replikasyon kullanıcısı — primary/setup-replication.sh

```
#!/bin/bash
set -e

psql -v ON_ERROR_STOP=1 --username "$POSTGRES_USER" --dbname "$POSTGRES_DB" <<'SQL'
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'replica123';
SQL

# Eğitim ağı için izin; üretimde CIDR daraltılmalıdır.
echo 'host replication replicator 0.0.0.0/0 scram-sha-256' >> "$PGDATA/pg_hba.conf"
```

Bu dosya PostgreSQL resmi image'ının /docker-entrypoint-initdb.d/ mekanizmasıyla ilk başlatmada çalışır. pg_hba.conf Replica'nın Primary'ye replikasyon bağlantısını yetkilendirir. replica123 sadece yerel laboratuvar parolasıdır.

7. Replica Dockerfile — replica/Dockerfile

```
FROM postgres:17
COPY replica-entrypoint.sh /usr/local/bin/replica-entrypoint.sh
RUN chmod +x /usr/local/bin/replica-entrypoint.sh
USER postgres
ENTRYPOINT ["/usr/local/bin/replica-entrypoint.sh"]
```

8. Replica başlatma script'i — replica/replica-entrypoint.sh

```
#!/bin/bash
set -euo pipefail

export PGDATA="${PGDATA:-/var/lib/postgresql/data}"
export PGPASSWORD="${REPLICATION_PASSWORD:?REPLICATION_PASSWORD missing}"
mkdir -p "$PGDATA"
chmod 700 "$PGDATA"

if [ ! -s "$PGDATA/PG_VERSION" ]; then
  echo 'Replica ilk kez kuruluyor: Primary bekleniyor...'
  until pg_basebackup \
      -h postgres-primary -p 5432 \
      -U replicator -D "$PGDATA" \
      -Fp -Xs -P -R; do
    echo 'Primary/replikasyon hazır değil; 3 saniye sonra tekrar...'
    # Başarısız denemeden kalan kısmi dosyaları temizle.
    find "$PGDATA" -mindepth 1 -maxdepth 1 -exec rm -rf {} +
    sleep 3
  done
fi

unset PGPASSWORD
exec postgres -c hot_standby=on
```

pg_basebackup -R, Replica için standby.signal ve primary_conninfo ayarlarını hazırlar. Replikasyon bağlantı parolası bu örnekte PGPASSWORD üzerinden pg_basebackup için sağlanır; yeniden başlatma sonrasında WAL

streaming'in devam edebilmesi için aşağıdaki Compose'da ayrıca PGPASSFILE tanımlayacağız ve bir parola dosyası oluşturacağız.

Önemli düzeltme: Script'te pg_basebackup çalışmadan önce ve PostgreSQL her yeniden başladığında kullanılacak .pgpass dosyasını da oluşturmalıyız. Yukarıdaki script'te mkdir -p "$PGDATA" satırından hemen önce aşağıdaki satırları ekle:

```
printf 'postgres-primary:5432:replication:replicator:%s\n' "$REPLICATION_PASSWORD" > /tmp/replica.pgpass
chmod 600 /tmp/replica.pgpass
export PGPASSFILE=/tmp/replica.pgpass
```

unset PGPASSWORD satırından sonra PGPASSFILE ortam değişkeni kalır; PostgreSQL standby bağlantısında dosyayı kullanabilir. Daha güvenli bir üretim uygulamasında bu parola Docker secret olarak yönetilmelidir.

9. Python bağımlılıkları — python/requirements.txt

```
psycopg2-binary==2.9.10
```

10. Python Dockerfile — python/Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

11. Python READ/WRITE algoritması — python/app.py

```
import os
from collections import defaultdict
from decimal import Decimal
import psycopg2

COMMON = dict(
    dbname="ecommerce_db",
    user="ecommerce_user",
    password=os.environ["DB_PASSWORD"],
    port=5432,
    connect_timeout=10,
)

read_conn = psycopg2.connect(host="postgres-replica", **COMMON)
try:
    with read_conn.cursor() as cur:
        cur.execute("SELECT order_id, quantity, unit_price, order_date FROM orders")
        orders = cur.fetchall()
finally:
    read_conn.close()

summary = defaultdict(lambda: [Decimal("0"), 0, 0])
for order_id, quantity, price, date in orders:
```

```
    summary[date][0] += quantity * price
    summary[date][1] += quantity
    summary[date][2] += 1

write_conn = psycopg2.connect(host="postgres-primary", **COMMON)
try:
    with write_conn:
        with write_conn.cursor() as cur:
            cur.execute("DELETE FROM daily_sales_summary")
            for date, (revenue, qty, count) in sorted(summary.items()):
                avg = (revenue / count).quantize(Decimal("0.01"))
                cur.execute("""
                    INSERT INTO daily_sales_summary
                    (sale_date, total_revenue, total_quantity,
                     order_count, average_order_value)
                    VALUES (%s, %s, %s, %s, %s)
                """, (date, revenue, qty, count, avg))
finally:
    write_conn.close()

print(f"Replica'dan {len(orders)} sipariş okundu.")
print(f"Primary'ye {len(summary)} günlük özet yazıldı.")
```

Bu uygulama bir batch job'dır: çalışır, hesaplar, yazar ve kapanır. Yeni siparişlerden sonra yeniden çalıştırılır. Replica gecikmesi varsa en güncel veriler henüz okunmayabilir; canlı sistemlerde replikasyon gecikmesi ölçülmeli veya veri tutarlılığı için ek önlemler alınmalıdır.

12. Grafana veri kaynağı — grafana/provisioning/datasources/postgres.yaml

```
apiVersion: 1
datasources:
  - name: Ecommerce-Replica
    uid: ecommerce_replica
    type: postgres
    access: proxy
    url: postgres-replica:5432
    user: ecommerce_user
    secureJsonData:
      password: ecommerce123
    jsonData:
      database: ecommerce_db
      sslmode: disable
      postgresVersion: 1700
      timescaledb: false
    isDefault: true
```

Grafana'nın veri kaynağı sadece Replica'yı kullanır. Eğitim ortamı dışına çıkarırken read-only kullanıcı ve güvenli secret yönetimi kullan.

13. Docker Compose — compose.yaml (tam dosya)

```
name: ecommerce-replica-lab

services:
  postgres-primary:
    image: postgres:17
    restart: unless-stopped
    environment:
      POSTGRES_USER: ecommerce_user
      POSTGRES_PASSWORD: ecommerce123
      POSTGRES_DB: ecommerce_db
    command:
      - postgres
      - -c
      - wal_level=replica
      - -c
      - max_wal_senders=10
      - -c
      - wal_keep_size=256MB
      - -c
      - listen_addresses=*
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - primary_data:/var/lib/postgresql/data
      - ./primary/init.sql:/docker-entrypoint-initdb.d/01-init.sql:ro
      - ./primary/setup-replication.sh:/docker-entrypoint-initdb.d/02-replication.sh:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ecommerce_user -d ecommerce_db"]
      interval: 5s
      timeout: 5s
      retries: 15

  postgres-replica:
    build: ./replica
    restart: unless-stopped
    environment:
      PGDATA: /var/lib/postgresql/data
      REPLICATION_PASSWORD: replica123
    ports:
      - "127.0.0.1:5433:5432"
    volumes:
      - replica_data:/var/lib/postgresql/data
    depends_on:
      postgres-primary:
```

```
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ecommerce_user -d ecommerce_db"]
      interval: 5s
      timeout: 5s
      retries: 20

  python:
    build: ./python
    environment:
      DB_PASSWORD: ecommerce123
    depends_on:
      postgres-replica:
        condition: service_healthy
    profiles: ["job"]

  grafana:
    image: grafana/grafana:12.2.0
    restart: unless-stopped
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: admin123
    ports:
      - "3000:3000"
    volumes:
      - grafana_data:/var/lib/grafana
      - ./grafana/provisioning/datasources:/etc/grafana/provisioning/datasources:ro
    depends_on:
      postgres-replica:
        condition: service_healthy

volumes:
  primary_data:
  replica_data:
  grafana_data:
```

profiles: ["job"] sayesinde Python sürekli servis olarak açılmaz; docker compose run --rm python ile isteğe bağlı çalışır.

14. Başlatma ve ilk kontroller

```
cd ~/ecommerce-replica
sudo docker compose config
sudo docker compose up -d --build
sudo docker compose ps
sudo docker compose logs --tail=80 postgres-replica
```

İlk replikasyon kurulumu birkaç dakika sürebilir. Replica hata verirse logları incele; hemen volume silme.

15. Streaming Replication gerçekten çalışıyor mu?

Primary üzerinde:

```
sudo docker compose exec postgres-primary psql -U ecommerce_user -d ecommerce_db -c "SELECT application_name,
client_addr, state, sync_state FROM pg_stat_replication;"
```

state alanında streaming beklenir. sync_state asenkron kurulumda async olabilir.

Replica üzerinde:

```
sudo docker compose exec postgres-replica psql -U ecommerce_user -d ecommerce_db -c "SELECT pg_is_in_
recovery();"
```

Beklenen: t (true). Primary üzerinde aynı sorgu f döndürür.

Replica'nın yazmaya kapalı olduğunu doğrula:

```
sudo docker compose exec postgres-replica psql -U ecommerce_user -d ecommerce_db -c "INSERT INTO
orders(product_name,quantity,unit_price,order_date) VALUES ('Test',1,1,CURRENT_DATE);"
```

Beklenen: cannot execute INSERT in a read-only transaction benzeri hata. Bu testte hata alınması başarıdır.

16. Python algoritmasını çalıştır

```
sudo docker compose build python

sudo docker compose run --rm python
```

Beklenen:

```
Replica'dan 10 sipariş okundu.

Primary'ye 5 günlük özet yazıldı.
```

Primary sonucu:

```
sudo docker compose exec postgres-primary psql -U ecommerce_user -d ecommerce_db -c "SELECT * FROM daily_
sales_summary ORDER BY sale_date;"
```

Beklenen toplamlar (TL): 20 Ekim 52500; 21 Ekim 20500; 22 Ekim 29800; 23 Ekim 12000; 24 Ekim 12600. Genel toplam 127400 TL, 10 sipariş, 35 ürün.

Replica sonucu:

```
sudo docker compose exec postgres-replica psql -U ecommerce_user -d ecommerce_db -c "SELECT * FROM daily_
sales_summary ORDER BY sale_date;"
```

Aynı sonuçlar kısa süre sonra Replica'da da görünmelidir.

17. Grafana dashboard

Ubuntu VM içindeki tarayıcıdan http://localhost:3000 aç. Kullanıcı: admin, parola: admin123. İlk girişte değiştirmen önerilir. Connections > Data sources altında otomatik oluşturulmuş Ecommerce-Replica veri kaynağını kontrol et ve Save & test kullan.

Dashboards > New > New dashboard > Add visualization ile PostgreSQL kaynağını seç. SQL düzenleyicisinde Code moduna geç. Dashboard zaman aralığını örnek veriler için 20–25 Ekim 2026 olarak ayarla.

Panel A — Günlük ciro (Time series)

```
SELECT sale_date AS "time", total_revenue AS "Günlük Ciro"

FROM daily_sales_summary

WHERE $__timeFilter(sale_date)

ORDER BY sale_date;
```

Panel B — Toplam ciro (Stat)

```
SELECT COALESCE(SUM(total_revenue),0) AS "Toplam Ciro"

FROM daily_sales_summary
```

```
WHERE $__timeFilter(sale_date);
```

Panel C — Sipariş sayısı (Stat)

```
SELECT COALESCE(SUM(order_count),0) AS "Sipariş Sayısı"
FROM daily_sales_summary
WHERE $__timeFilter(sale_date);
```

Panel D — Günlük özet (Table)

```
SELECT sale_date AS "Tarih", total_revenue AS "Ciro",
       total_quantity AS "Ürün Adedi", order_count AS "Sipariş",
       average_order_value AS "Ortalama Sipariş"
FROM daily_sales_summary
WHERE $__timeFilter(sale_date)
ORDER BY sale_date;
```

Grafana para alanlarının Unit ayarını TRY/Türk lirası yapabilirsin. Dashboard'u kaydet.

18. Uçtan uca güncelleme testi

Yeni siparişi yalnızca Primary'ye ekle:

```
sudo docker compose exec postgres-primary psql -U ecommerce_user -d ecommerce_db -c "INSERT INTO
orders(product_name,quantity,unit_price,order_date) VALUES ('Laptop',3,30000,'2026-10-24');"
```

Replica'da 11 sipariş görününceye kadar kısa süre bekle ve kontrol et:

```
sudo docker compose exec postgres-replica psql -U ecommerce_user -d ecommerce_db -c "SELECT COUNT(*) FROM
orders;"
```

Ardından:

```
sudo docker compose run --rm python
```

24 Ekim toplamı 102600 TL, ürün adedi 12, sipariş sayısı 3, ortalama sipariş 34200 TL olmalıdır. Grafana'da dashboard'u yenile; güncelleme Replica'ya aktarıldıktan sonra görünür.

19. Hata giderme

- Port already allocated: sudo ss -lntp | grep -E ':(5432|5433|3000)\b'; eski proje ya da host PostgreSQL/Grafana çalışıyor olabilir.
- Replica bağlanamıyor: sudo docker compose logs postgres-primary postgres-replica; Primary'de pg_stat_replication, Replica'da pg_is_in_recovery() kontrol et.
- Replication authentication failed: replicator kullanıcısı, pg_hba.conf, parola ve ilk kurulum loglarını kontrol et.
- Replica'da boş tablo: Replikasyonun streaming olduğuna ve sorguladığın veritabanının ecommerce_db olduğuna bak.
- Grafana veri yok: Kaynak postgres-replica:5432 olmalı; zaman aralığı 20–25 Ekim 2026 olmalı.
- Python hata veriyor: sudo docker compose run --rm python çıktısını incele; Replica sağlıklı mı kontrol et.
- pg_basebackup başarısız: Replica volume'u kısmen dolmuş olabilir. Script başarısız denemelerde temizler; Primary kullanıcı/erişim sorunu çözülmeden tekrar denemek işe yaramaz.
- Eski data ile çakışma: name: ecommerce-replica-lab yeni volume'ları ayrı tutar. Eski projede container_name kullanılmış olsa bile down sonrası çakışma beklenmez.

20. Günlük yönetim ve temizlik

```
cd ~/ecommerce-replica
sudo docker compose up -d
sudo docker compose ps
```

```
sudo docker compose logs -f postgres-replica

sudo docker compose run --rm python

sudo docker compose down
```

down verileri korur; down -v bu yeni projenin Primary, Replica ve Grafana volume'larını siler. Yalnızca tüm verileri bilinçli olarak sıfırlamak istediğinde kullan.

21. Son kontrol listesi

- [ ] Eski proje yedeklendi ve durduruldu.
- [ ] Primary 5432, Replica 5433, Grafana 3000 host portlarında.
- [ ] Primary'de pg_stat_replication.state = streaming.
- [ ] Replica'da pg_is_in_recovery() = true.
- [ ] Replica'ya INSERT başarısız (read-only).
- [ ] Python Replica'dan okuyup Primary'ye özet yazıyor.
- [ ] Grafana Ecommerce-Replica kaynağını kullanıyor.
- [ ] Yeni sipariş testi dashboard'a yansıyor.

22. Teknik notlar ve resmi kaynaklar

- PostgreSQL Streaming Replication: https://www.postgresql.org/docs/17/warm-standby.html#STREAMING-REPLICATION
- PostgreSQL pg_basebackup: https://www.postgresql.org/docs/17/app-pgbasebackup.html
- PostgreSQL pg_hba.conf: https://www.postgresql.org/docs/17/auth-pg-hba-conf.html
- PostgreSQL resmi Docker image: https://hub.docker.com/_/postgres
- Docker Compose: https://docs.docker.com/compose/
- Grafana PostgreSQL: https://grafana.com/docs/grafana/latest/datasources/postgres/

Not: Bu, yerel laboratuvar projesidir. Parolalar örnek amaçlı açık yazılmıştır. Üretimde secrets, TLS, dar ağ izinleri, izleme, yedekleme, failover ve kapasite planlaması gerekir.